

DOCUMENTAÇÃO TÉCNICA

ReGraphik — Sistema de Gestão de Estoque Reverso

Projeto:	ReGraphik — TCC Técnico em Desenvolvimento de Sistemas
Instituição:	SENAI
Autores:	Bruno Maia, Otávio Henrique, Lucas Aquino, Luna Beatriz, Kaio Alves
Ano:	2025
Repositório:	github.com/BrunoMaiaSenai/ReGraphikApp
API Base URL:	https://webregraphik.runasp.net
Documento:	Documentação Técnica — Versão 1.0

1. Visão Geral do Sistema

O ReGraphik é um sistema de gestão de estoque reverso voltado ao setor gráfico. Sua arquitetura é composta por dois subsistemas integrados: uma API REST em ASP.NET Core responsável pelo processamento de regras de negócio e persistência no Firebase Realtime Database; e um cliente desktop WPF (.NET) estruturado no padrão MVVM. A integração com a Google Maps Places API permite localização georreferenciada de pontos de coleta seletiva.

1.1 Stack Tecnológico

Camada	Tecnologia	Versão / Detalhe
Linguagem	C# (.NET)	.NET 8 / .NET 9
Frontend Desktop	WPF — Windows Presentation Foundation	Windows-only
Padrão de Projeto	MVVM (Model-View-ViewModel)	Padrão GoF
Backend / API	ASP.NET Core Web API	REST / JSON
Banco de Dados	Firebase Realtime Database	NoSQL — Google Cloud
Autenticação Firebase	Google Service Account (JSON)	OAuth 2.0
Mapas / Geolocalização	Google Maps Places API + Leaflet.js	WebBrowser/WebView2
Documentação da API	Swagger / OpenAPI	Swashbuckle
Controle de Versão	Git / GitHub	214+ commits
IDE de Desenvolvimento	Visual Studio 2022	Community Edition

1.2 Requisitos Mínimos de Hardware e Software

- Processador: Intel Core i3 ou equivalente (2 GHz ou superior)
- Memória RAM: 4 GB (8 GB recomendado para ambiente de desenvolvimento)
- Armazenamento: 500 MB para instalação (dados persistidos em nuvem no Firebase)
- Sistema Operacional: Windows 10 / Windows 11 (64 bits)
- Conexão com a internet: necessária para Firebase e Google Maps Places API
- Software: .NET 8 Runtime, Visual C++ Redistributable, WebView2

2. Arquitetura do Sistema

2.1 Visão Geral em Camadas

O sistema é composto por três camadas principais:

1. **Cliente WPF (Frontend Desktop)** — interface do usuário construída em XAML/C# com padrão MVVM.

2. **API REST em ASP.NET Core (Backend)** — processamento de regras de negócio e exposição de endpoints HTTP.

3. **Firebase Realtime Database (Persistência)** — banco de dados NoSQL em nuvem (Google).

Fluxo de comunicação entre camadas:

```

WPF CLIENT (View → ViewModel → Command)
|
| HTTP / REST (JSON)
↓
API REST – ApiRestReGraphik (ASP.NET Core)
Controllers → Services → Firebase SDK
|
| Firebase SDK (HTTPS)
↓
FIREBASE REALTIME DATABASE (NoSQL – Google Cloud)
|
| Google Maps Places API (HTTPS)
↓
GOOGLE MAPS PLACES API (NearbySearch / TextSearch)
  
```

2.2 Padrão MVVM no Cliente WPF

O cliente desktop segue o padrão MVVM (Model-View-ViewModel), com separação clara de responsabilidades:

Camada	Componentes	Responsabilidade
VIEW	MainWindow, DashboardPage, ResiduosPage, EstoqueReversoPage, MapaPage, RelatoriosPage (XAML)	Apresentação visual; bindings declarativos; zero lógica de negócio; zero code-behind.
VIEWMODEL	BaseViewModel, ResiduoViewModel, LoginViewModel, MapViewModel (C#)	Lógica de apresentação; implementa INotifyPropertyChanged; expõe ObservableCollections e RelayCommand.
MODEL	Usuario, Residuo, Sugestao, SugestaoResiduo, PontosColeta (POCOs C#)	Entidades do domínio puras, sem dependência de UI ou infraestrutura.
SERVICE LAYER	GooglePlacesService, ApiService (C#)	Lógica HTTP, integração com API REST e Google Maps; injetado nos ViewModels via construtor.

2.2.1 BaseViewModel e INotifyPropertyChanged

Todos os ViewModels herdam de BaseViewModel, que implementa INotifyPropertyChanged. Isso permite que a View reaja automaticamente a mudanças de propriedades via Data Binding, sem acoplamento direto entre as camadas.

2.2.2 RelayCommand

O padrão Command é implementado por RelayCommand, que encapsula delegates Action e Func para execução e validação de comandos. Cada botão da interface é associado a um Command via binding XAML, eliminando o uso de event handlers no code-behind.

3. Estrutura do Repositório

```
ReGraphikApp/
├── ApiRestReGraphik/ # Backend – ASP.NET Core REST API
│   ├── Controllers/ # Endpoints HTTP (CRUD completo)
│   ├── Services/ # Regras de negócio e acesso ao Firebase
│   ├── Models/ # Entidades do domínio
│   ├── Data/ # Configuração Firebase (DbReGraphik.cs)
│   └── Program.cs # DI, Swagger, CORS
├── ReGraphik/ # Frontend – WPF Desktop (MVVM)
│   ├── Views/Pages/ # Dashboard, Resíduos, Mapa, Relatórios
│   ├── ViewModels/ # BaseViewModel, ResiduoViewModel
│   ├── Models/ # Espelho das entidades do domínio
│   ├── Services/ # GooglePlacesService
│   └── Commands/ # RelayCommand
└── Modelagem/ # PDFs: Conceitual, Lógico, Físico, MVVM
```

4. Modelagem do Banco de Dados

4.1 Entidades e Relacionamentos

O modelo relacional é composto por cinco entidades principais:

Relacionamento	Cardinalidade	Semântica
USUARIO — CADASTRO_RESIDUOS	1 : N	Um usuário pode cadastrar vários resíduos.
TIPO_MATERIAL — CADASTRO_RESIDUOS	1 : N	Um tipo de material classifica vários resíduos.
TIPO_MATERIAL — SUGESTOES	1 : N	Um tipo de material pode ter várias sugestões.
CADASTRO_RESIDUOS — SUGESTOES	1 : N	Resolvido por SUGESTOES_RESIDUOS: um resíduo recebe várias sugestões.

4.2 Tabela: CadastroUsuarios

Coluna	Chave	Tipo	Descrição
ID	PK	INT / String	Identificador único (GUID)
Nome	—	VARCHAR(100)	Nome completo do usuário
CPF	—	VARCHAR(14)	CPF único do usuário
Email	—	VARCHAR(150)	E-mail de contato
Login	—	VARCHAR(50)	Login de acesso (único)

Senha	—	VARCHAR(255)	Hash da senha
DataCadastro	—	DATETIME	Data/hora de cadastro (UTC)

4.3 Tabela: TipoMaterial

Coluna	Chave	Tipo	Descrição
Id	PK	INT / String	Identificador único
DescricaoMaterial	—	VARCHAR(100)	Descrição do tipo (papel, vinil, cartão, etc.)

4.4 Tabela: CadastroResiduos

Coluna	Chave / Relacionamento	Tipo	Descrição
ID	PK	INT / String	Identificador único
IdUsuario	FK → CadastroUsuarios(ID)	INT / String	Usuário cadastrante
IdTipoMaterial	FK → TipoMaterial(Id)	INT / String	Tipo do material
Origem	—	VARCHAR(100)	Origem do resíduo (projeto/setor)
Especificacao	—	TEXT	Especificação técnica
Projeto	—	VARCHAR(100)	Projeto de origem
Quantidade	—	DECIMAL	Quantidade disponível em estoque
DataCadastro	—	DATETIME	Data/hora de cadastro
Condicao	—	VARCHAR(50)	Estado físico do material
DimensoesCm	—	VARCHAR(50)	Dimensões (cm)
DimensoesLm	—	VARCHAR(50)	Dimensões (metros lineares)
Observacao	—	TEXT	Observações livres
Anexo	—	VARCHAR(255)	Caminho do arquivo anexado
Status	—	ENUM	Em Estoque Reaproveitado Descartado

4.5 Tabela: Sugestoes

Coluna	Chave	Tipo	Descrição
Id	PK	INT / String	Identificador único
IdTipoMaterial	FK → TipoMaterial(Id)	INT / String	Tipo de material relacionado
DescricaoSugestao	—	TEXT	Descrição da sugestão de reaproveitamento

4.6 Tabela: SugestoesResiduos

Coluna	Chave	Tipo	Descrição
ID	PK	INT / String	Identificador único
IdCadastroResiduo	FK → CadastroResiduos(ID)	INT / String	Resíduo vinculado
IdSugestao	FK → Sugestoes(Id)	INT / String	Sugestão aplicada
DataAplicacao	—	DATETIME	Data em que a sugestão foi aplicada

5. API REST — Endpoints e Integração

A API REST do ReGraphik é desenvolvida em ASP.NET Core e exposta no domínio <https://webregraphik.runasp.net>. Documentação Swagger disponível em `/swagger`.

5.1 Endpoints Principais

Método	Endpoint	Descrição
GET	/api/residuos	Lista todos os resíduos com filtros opcionais de status e tipo
GET	/api/residuos/{id}	Retorna resíduo específico com histórico de sugestões aplicadas
POST	/api/residuos	Cadastra novo resíduo — valida tipo, dimensões e usuário autenticado
PUT	/api/residuos/{id}/status	Atualiza status (ativo, em_uso, descartado) com log de alteração
DELETE	/api/residuos/{id}	Remove resíduo — requer perfil ADMIN, registra auditoria
GET	/api/usuarios	Lista usuários — restrito ao perfil ADMIN
POST	/api/sugestoes/aplicar	Aplica sugestão de reaproveitamento — gera registro SugestaoResiduo
GET	/api/coleta/proximos	Integra GooglePlacesService — retorna pontos por lat/lng

5.2 Controller: api/Usuario

Método	Rota	Descrição
GET	/api/Usuario	Lista todos os usuários
GET	/api/Usuario/{id}	Obtém usuário por ID
POST	/api/Usuario	Cria novo usuário
POST	/api/Usuario/login	Autenticação com login + senha
POST	/api/Usuario/finalizar-cadastro?token=...	Finaliza pré-cadastro com token autorizado
PUT	/api/Usuario/{id}	Atualiza usuário existente
DELETE	/api/Usuario/{id}	Remove usuário

5.3 Controller: api/PontosColeta

Método	Rota	Descrição
GET	/api/PontosColeta	Lista todos os pontos cadastrados no Firebase
GET	/api/PontosColeta/{id}	Obtém ponto por ID
POST	/api/PontosColeta/google?cidade=...	Cadastra pontos via Google Maps Places API e salva no Firebase
POST	/api/PontosColeta	Cadastra novo ponto de coleta manualmente

PUT	/api/PontosColeta/{id}	Atualiza ponto existente
DELETE	/api/PontosColeta/{id}	Remove ponto de coleta

O endpoint /google valida se a cidade já está cadastrada no Firebase antes de consultar o Google Maps. Se nova, busca os pontos, salva automaticamente e retorna a lista com IDs gerados.

5.4 Controller: api/Sugestao

Método	Rota	Descrição
GET	/api/Sugestao	Lista todas as sugestões
GET	/api/Sugestao/{id}	Obtém sugestão por ID
POST	/api/Sugestao	Cria nova sugestão
PUT	/api/Sugestao/{id}	Atualiza sugestão existente
DELETE	/api/Sugestao/{id}	Remove sugestão

5.5 Controller: api/SugestaoResiduos

Método	Rota	Descrição
GET	/api/SugestaoResiduos	Lista todas as sugestões aplicadas a resíduos
GET	/api/SugestaoResiduos/{id}	Obtém por ID
POST	/api/SugestaoResiduos	Registra aplicação de sugestão a um resíduo
PUT	/api/SugestaoResiduos/{id}	Atualiza registro
DELETE	/api/SugestaoResiduos/{id}	Remove registro

5.6 Fluxo Detalhado — Busca de Pontos de Coleta

1. Usuário clica em "Buscar" na MapaPage (View XAML).
2. O evento aciona BtnBuscar_Click no code-behind, que monta a URL da API com a cidade informada.
3. A API recebe a requisição e constrói a query para a Google Maps Places API.
4. O resultado retornado pelo Google Maps é persistido no Firebase (pontos_coleta).
5. A API retorna a lista de pontos (lat/lng) como JSON para o cliente WPF.
6. O cliente WPF gera o HTML com Leaflet.js, grava em arquivo temporário e renderiza no WebBrowser.

6. Requisitos do Sistema

6.1 Requisitos Funcionais

Código	Descrição
RF-001	Autenticação e Controle de Acesso: login com usuário e senha; diferenciação de perfis comum e Administrador.
RF-002	Cadastro e Triagem de Resíduos: registro detalhado com especificação, origem, quantidade, dimensões e anexo fotográfico.
RF-003	Atualização Automática de Status: controle do estado entre Em Estoque, Reaproveitado e Descartado.
RF-004	Motor de Sugestões de Reaproveitamento: correlação automática de sugestões com base no tipo de material.
RF-005	Geolocalização de Pontos de Coleta: busca por cidade consumindo dados georreferenciados em tempo real.
RF-006	Sincronização com Firebase: novos pontos de coleta salvos automaticamente para consultas futuras.
RF-007	Auditoria e Exclusão Segura: remoção restrita a administradores com trilha de auditoria automática.
RF-008	Geração de Relatórios e Dashboards: KPIs visuais e exportação de dados tabulados em CSV.

6.2 Requisitos Não Funcionais

Código	Descrição
RNF-001	Desempenho: API REST usa async/await para não bloquear a thread principal do cliente.
RNF-002	Compatibilidade: cliente frontend funciona em Windows 10/11 (64 bits) com .NET 8 Runtime.
RNF-003	Disponibilidade: Firebase Realtime Database garante SLA conforme plano de serviço.
RNF-004	Escalabilidade: padrão MVVM desacoplado permite substituição de serviços via injeção de dependências.
RNF-005	Renderização de Mapas: Leaflet.js carregado via WebView2/WebBrowser sem licenciamento adicional.
RNF-006	Segurança: senhas armazenadas apenas como hash criptografado, nunca em texto claro.

7. Módulos do Sistema

Módulo	Descrição	Status
Mapa / Pontos de Coleta	Integração com Google Places API para localização de pontos de coleta próximos	Disponível
Dashboard	KPIs e indicadores de estoque reverso em tempo real via Firebase	Em Construção
Cadastro de Resíduo	Triagem, classificação e registro de materiais com suporte a anexos	Em Construção
Sugestões	Motor de recomendação de reaproveitamento por tipo de material	Em Construção
Relatórios	Exportação de histórico e dados operacionais (CSV/PDF)	Em Construção
Chat Interno	Canal de comunicação entre usuários da plataforma (Firebase)	Implementação Futura
Central de Notificações	Alertas automáticos de estoque e pendências operacionais	Implementação Futura
Log de Consulta	Auditoria e rastreabilidade completa de operações no sistema	Implementação Futura

8. Processo de Desenvolvimento

Fase	Commits	Atividades	Status
Fase 1 — Setup	30	Estrutura da solução C#, BaseViewModel, INotifyPropertyChanged, RelayCommand, modelagem in	Concluída
Fase 2 — Core UI	80	WPF XAML (layout principal), tela de Login + ViewModel, Mapa do GooglePlacesService, início do C	Concluída
Fase 3 — API REST	73+	ApiRestReGraphik criado, Controllers (Resíduos, Usuários, Sugestões), integração API ↔ WPF Clie	Em andamento

Total de commits: **214+** até a entrega desta documentação. Repositório: github.com/BrunoMaiaSenai/ReGraphikApp.

9. Viabilidade Técnica

9.1 Componentes e Licenças

Componente	Licença	Maturidade	Observação
C# / .NET 8+	MIT (open-source)	Alta	Plataforma Microsoft com suporte LTS garantido
ASP.NET Core	MIT (open-source)	Alta	Framework web maduro, utilizado em produção globalmente
WPF	MIT (open-source)	Alta	Framework nativo Windows com base de código estável
Firebase Realtime DB	Gratuito (plano Spark)	Alta	Google Cloud com 99,95% de SLA
Google Maps Places API	Gratuito até US\$200/mês	Alta	Crédito cobre ~13.000 buscas/mês
Leaflet.js	BSD-2-Clause	Alta	Biblioteca de mapas mais utilizada do mundo
Swagger/OpenAPI	Apache 2.0	Alta	Padrão de mercado para documentação de APIs REST

Git / GitHub	Gratuito (plano Free)	Alta	Plataforma de controle de versão líder de mercado
--------------	-----------------------	------	---

9.2 Riscos Técnicos e Mitigações

Risco	Prob.	Impacto	Mitigação
Cota gratuita do Firebase excedida	Média	Alto	Migrar para plano Blaze (pay-as-you-go) ou adotar banco relacional próprio
Cota gratuita do Google Maps excedida	Média	Médio	Implementar cache de resultados no Firebase e limitar chamadas por sessão
Dependência exclusiva de Windows (WPF)	Baixa	Médio	Desenvolver versão web (Blazor) ou mobile (.NET MAUI) em iterações futuras
Ausência de autenticação JWT na API	Alta	Alto	Implementar JWT como prioridade na próxima sprint (já no roadmap)
Perda de conectividade com a internet	Baixa	Alto	Implementar cache local (SQLite) para funcionamento offline parcial
Quebra de contrato da Google Maps API	Baixa	Médio	Interface IPlacesService abstrai o provedor, permitindo substituição sem impacto

10. Viabilidade Econômica

10.1 Custos de Desenvolvimento (Projetados)

Item	Horas Est.	Valor/Hora	Custo Total
Levantamento de requisitos e Mini Mundo	20 h	R\$ 60,00	R\$ 1.200,00
Modelagem de dados (Conceitual, Lógico, Físico)	30 h	R\$ 60,00	R\$ 1.800,00
Desenvolvimento da API REST (ASP.NET Core)	80 h	R\$ 80,00	R\$ 6.400,00
Desenvolvimento do cliente WPF (MVVM)	100 h	R\$ 80,00	R\$ 8.000,00
Integração Firebase + Google Maps API	40 h	R\$ 80,00	R\$ 3.200,00
Testes, correções e refatoração	30 h	R\$ 60,00	R\$ 1.800,00
Documentação técnica e TCC	30 h	R\$ 50,00	R\$ 1.500,00
TOTAL	330 h	—	R\$ 23.900,00

10.2 Custos Operacionais Mensais (Estimativa)

Serviço	Custo Mensal Estimado	Observação
Firebase Realtime DB (plano Spark)	R\$ 0,00	Gratuito até 1 GB armazenamento e 10 GB/mês transferência
Firebase Realtime DB (plano Blaze)	R\$ 25,00 – R\$ 150,00	Pay-as-you-go acima do plano gratuito
Google Maps Places API	R\$ 0,00 – R\$ 100,00	Crédito de US\$200/mês cobre ~13.000 buscas
Hospedagem da API	R\$ 0,00 – R\$ 80,00	Render / Railway / Azure Free (planos gratuitos disponíveis)
Domínio e SSL (opcional)	R\$ 5,00 – R\$ 40,00	Let's Encrypt fornece SSL gratuito
TOTAL ESTIMADO	R\$ 0,00 – R\$ 370,00/mês	Cenário 100% gratuito viável para pequeno porte

10.3 Análise Custo-Benefício Anual

Categoria / Item	Tipo	Valor Anual Estimado
Desenvolvimento (amortizado em 3 anos)	Custo	R\$ 7.967,00/ano
Operação / Infraestrutura (cenário básico)	Custo	R\$ 0,00 – R\$ 1.200,00/ano
Manutenção e suporte técnico	Custo	R\$ 1.200,00 – R\$ 2.400,00/ano
Redução de custos com descarte	Benefício	R\$ 7.200,00 – R\$ 10.800,00/ano
Receita com venda de recicláveis	Benefício	R\$ 3.600,00 – R\$ 19.200,00/ano
Economia em matéria-prima (reaproveitamento)	Benefício	R\$ 2.400,00 – R\$ 6.000,00/ano

SALDO LÍQUIDO ESTIMADO

Lucro

R\$ 4.033,00 – R\$ 24.433,00/ano

11. Artefatos Técnicos Entregues

Artefato	Descrição	Status
Mini Mundo da Demanda	Contextualização do problema, atores e regras de negócio	Entregue
Modelo Conceitual (DER)	Diagrama ER com 5 entidades e seus relacionamentos (BRModelo)	Entregue
Modelo Lógico	Esquema relacional normalizado com PK, FK e restrições de integridade	Entregue
Modelo Físico	Script SQL completo com tabelas, índices e triggers	Entregue
Swagger / OpenAPI	Documentação interativa de todos os endpoints REST da API	Entregue
Repositório GitHub	Código-fonte completo com histórico de 182+ commits	Entregue
Apresentação MVVM	Slides técnicos detalhando arquitetura, padrões e integração	Entregue
Documentação TCC	Documentação completa do sistema para entrega do TCC	Entregue
Diagrama MVVM	Diagrama das ViewModels, bindings e Commands	Entregue

12. Referências

- MICROSOFT. ASP.NET Core documentation. <https://learn.microsoft.com/aspnet/core>. 2025.
- MICROSOFT. Windows Presentation Foundation. <https://learn.microsoft.com/dotnet/desktop/wpf>. 2025.
- GOOGLE. Firebase Realtime Database. <https://firebase.google.com/docs/database>. 2025.
- GOOGLE. Places API (New). <https://developers.google.com/maps/documentation/places/web-service>. 2025.
- AGAMBARD, T. et al. Leaflet.js. <https://leafletjs.com>. 2025.
- FOWLER, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
- MICROSOFT. Model-View-ViewModel (MVVM). <https://learn.microsoft.com/dotnet/architecture/maui/mvvm>. 2025.
- RICHARDSON, Leonard. RESTful Web Services. O'Reilly Media, 2007.
- SWAGGER. OpenAPI Specification. <https://swagger.io/specification>. 2025.
- BRMODELO. Ferramenta de Modelagem Entidade-Relacionamento. <https://www.sis4.com/brModelo>. 2025.